Clustering and classification for paediatric mTBI

Prashanth Velayudhan

5/4/2022

Table of contents

1 Home

This is a book created from markdown and executable code. Chapters of the book represent distinct steps of progress along my PhD work. Each chapter is a markdown file in the github repository psvelayudhan/research. Some chapters are rendered to include code. The included code is not actually present in this repository, but is being child-imported from R markdown files that are in other code repositories (as of 2022/05/28, all code is present in Eman's Wheeler Lab repo).[1]

Every time a quarto book (such as this) is rendered, the included code is executed. Because this book is expected to contain many analyses that read and write to many different parts of our lab's carbon share, the Rmarkdown files containing the included code are structured to not wreak havoc on every render. By default, all code chunks that do any 'heavy lifting' are set to not actually execute thanks to the `eval=FALSE` flag. Occasional chunks that do run are self-contained and are expected to simply read and display output generated by the serious chunks. For a user to actually execute any of the analyses displayed in this book, they must head on over to the actual source code and execute all those `eval=FALSE` chunks for themselves.

The short version: - This book provides a tidy way to track all the analyses I do for my PhD - The code in this book is always the up-to-date code used for all analyses - Rendering this book should not destroy our lab's carbon

[1] Not the same as sourcing R scripts. The Rmarkdown files are structured to be executed sequentially, and each script reads in the data it requires and writes out the data for subsequent analyses to reduce their inter-dependence.

2 Introduction

2.1 Incorporating pilot work:

- Keep the approach of bulk data grabbing scripts as done for the pilot
- Include a new section on data preprocessing for SNF (04-preprocessing for SNF)
- convert SNF script into a pilot script that explores 2 and 3 cluster options. this will become 05-pilot-snf and make room for 06-snf-approach-1 (inputs are inputs, outputs are not included - similar to what was done for the pilot)
- remove the old branch that was specifically for 3 cluster options
- new snf scripts can be used to explore the alternative styles of subtyping (e.g., including outputs in with the subtyping)
- package development can be the next major stage: a refactoring of the code present earlier in the process (abcdwheelR)
- systematic review on the horizon!

The introduction should include background information, rationale, current status of literature, hypotheses (?) and aims/objectives.

The logical flow of my research project:

1. Paediatric mTBI can lead to serious impairments
2. Heterogeneity is a barrier to mTBI care
3. Previous attempts at characterizing this heterogeneity were limited
4. We have lots of factors and a new way to subtype over all of them!

- New data sources in depth
- SNF in depth
- Previous subtyping attempts in depth
- We are poised to address the following gaps in knowledge and objectives:
  - are there long-term subtypes of mTBI?
  - how do they differ from the regular population
  - how do the subtypes change with time?
  - how effective are the clusters for prediction (predicting WHO gets mTBI, predicting what outcomes of mTBI you will have - ANOVA across individual measures OR clustered outcomes)
  - can we validate this in other datasets?

- what are these subtypes? classic characterization / identification methods

Concern: Our children did not all experience their mTBIs at the same time point. However, they are approximately equal in age. It is not practical to for a dataset to have a tight control on both time since injury and age, particularly outside of the context of school-level athletics. Our study assumes & tests the notion that there is heterogeneity at a persistent scale a

Mild traumatic brain injuries (mTBI) are highly prevalent in children, can result in serious and long-lasting impairments, and currently have no effective treatments. The heterogeneity of patient and injury factors is considered to be a major barrier to improving the clinical management of mTBIs. The first part of my proposed PhD work will be to apply similarity network fusion

2.2 Mild traumatic brain injury

Each **bookdown** chapter is an .Rmd file, and each .Rmd file can contain one (and only one) chapter. A chapter *must* start with a first-level heading: `# A good chapter`, and can contain one (and only one) first-level heading.

Use 2nd-level and higher headings within chapters like: `## A short section` or `### An even shorter section`.

The `index.Rmd` file is required, and is also your first book chapter. It will be the homepage when you render the book.

You can use anything that Pandoc's Markdown supports; for example, a math equation $a^2 + b^2 = c^2$.

It is also handy to account for good references, like what was done in Wang et al., 2014[1].

2.3 The Adolescent Brain Cognitive Development (ABCD) study

The Adolescent Brain Cognitive Development (ABCD) study is a long term study of pediatric development tracking patients from 21 research sites across the United States.

2.3.1 Identifying mTBI patients in the ABCD study

The ABCD study

2.4 Cluster analysis

2.4.1 Clustering techniques

2.4.2 Cluster analysis evaluation

2.4.2.1 Cluster tendency

Will not examine the Hopkins statistic for cluster tendency - "The relatively high value for H in this single-cluster case is due to a comparison of the structure of this data set so uniformly distributed random numbers, Le., data intended to be completely unstructured. With unstructured data as a comparison, virtually any structure in the test data would lead to rejection of the null hypothesis whether the data are clustered or not"**lawson1990?**. However even such a variation of the Hopkins statistic would not be so interesting, and would likely be expected - demographic data is random noise.

Whether or not the clusters are randomly distributed. This is challenging in the context of biomedical data - we do not expect the clusters to be randomly distributed by default (the demographic composition of any given country is far from random). In the context of mTBI, there are many facets of heterogeneity that would be interesting to examine:

- Heterogeneity of children with mTBI vs. those without
  - **Identifying the risk-of-sustaining-mtbi groups**: of all the children without a history of mTBI at baseline, is there a difference between the clustering of those that will develop an mTBI later on in the study and those who will not?
  - **Identifying the types of mTBI patients**: how does the heterogeneity of those who have mTBIs differ from the heterogeneity of those who don't have an mTBI? (Contrasted with the previous bullet point, this will help us learn more about the clusters that are formed BECAUSE of the mTBI and not because of being at a higher risk of mTBI)
    - **Identifying the types of mTBI outcomes:**
      - purely examining the clusters of patients using PPCS as the only input variable
      - can contrast mTBI PPCS vs. non-mTBI PPCS (though in that case, it's not quite a PPCS)

2.5 Similarity network fusion

Advances in technology have facilitated the collection of massive and diverse datasets. To harness the full potential of these datasets for pattern detection, prediction, and decision-making, techniques to effectively integrate diverse types of data are necessary.

Similarity network fusion (SNF), developed by Wang et al. in 2014[1], is an example of such a technique which works by first constructing similarity pairwise similarity matrices of all observations for each data type and then merging those matrices into a single network in a way that captures the full spectrum of the underlying data.

2.5.1 Advantages of SNF over other data-integration techniques

The most basic form of data-integration is to simply normalize and concatenate all available data types prior to analysis. This approach is known to reduce the signal-to-noise ratio of each individual data type[1], which is a serious concern in the context of biological data that is already very noisy to begin with. The authors of SNF also note that attempting to enhance signal by only including a pre-selected subset of data measures (features) can result in biased analyses.

Analysis of each data type prior to combination can be difficult when the results of the independent analyses don't agree with each other[1]. Circumventing this issue with consensus clustering (clustering based on the agreement of multiple different clustering algorithms) can result in the loss of valuable complementary information (i.e., the information that is not shared across data types), which is somewhat antithetical to the purpose of integrating diverse data-types in the first place.

2.5.2 Limitations of SNF

2.5.3 The SNF pipeline

2.5.3.1 The basic pipeline

The core procedure associated with SNF analyses involves the following steps:

1. Construct a similarity matrix for each pair of samples for each available data type
2. Fuse the similarity matrices using an iterative, nonlinear method (SNF)

User-defined parameters of the pipeline include:

- The data types to include
- The similarity measure to use for each data type

- Hyperparameters related to the fusion step (though the original SNF paper demonstrates strong robustness

The topic of feature selection (determining which variables to include) is not given much attention in the original description of the SNF pipeline. Though feature selection is known to be important in traditional cluster analysis due to the sensitivity of the algorithms to high-dimensionality,

2.5.3.2 The original glioblastoma multiforme case study

The authors of SNF work through a case study of applying the tool to the aggressive brain tumor, glioblastoma multiforme (GBM)[1].

2.5.4 The mathematical basis of SNF

2.6 Previous literature

2.7 Previous studies subtyping SNF

[2] applied SNF to integrate a variety of neuroimaging measures from children with neurodevelopmental disorders including autism spectrum disorder (ASD), obsessive-compulsive disorder (OCD), and attention-deficit/hyperactivity disorder (ADHD). Intriguingly, the derived clusters revealed that there were several children who were more similar to those of a different neurodevelopmental disorder than to those of their own.

The pipeline in this study consisted of

For aa right triangle, if c denotes the *length* of the hypotenuse and a and b denote the lengths of the **other** two sides, we have

$$a^2 + b^2 = c^2$$

Here is an equation.

$$f(k) = \binom{n}{k} p^k (1-p)^{n-k} (\#eq : binom) \tag{2.1}$$

You may refer to using `\@ref(eq:binom)`, like see Equation @ref(eq:binom).

3 Summary

In summary, this book has no content whatsoever.

4 Proposed methods

Here is where I will say what I plan on doing.

5 Expected outcomes and possible pitfalls

Here is what I will talk about the things I believe may happen.

6 Identifying subjects

The TBI information of the ABCD dataset is split across a few files:

- `abcd_otbi01.txt`: the raw, modified Ohio State TBI screen given at baseline
- `abcd_tbi01.txt`: summary scores of abcd_otbi01.txt
- `abcd_lpohstbi01.txt`: all longitudinal follow-ups of the baseline screen
- `abcd_lsstbi01.txt`: summary scores of abcd_lpohstbi01.txt

All information required for identifying children at a chronic stage following mTBI as of baseline data collection is available in the `abcd_otbi01.txt` file.

6.1 Renaming the mTBI data

The script below was used to rename the data columns of the abcd_otbi01 file, as the provided names are a bit tough to keep track of.

Script location:

```
[1] "wheeler-lab/abcd/prashanth-proj/00-rename-data/rename_abcd_otbi01.Rmd"
```

Renaming abcd_otbi01 columns

```
library(tidyverse)

abcd_otbi01 <- read_delim("lab-data/abcd/4.0/trauma/abcd_otbi01.txt")[-1, ]

abcd_otbi01_renamed <- abcd_otbi01 %>%
    rename(
        hosp_er_inj = tbi_1, # ever hospitalized/ER for head/neck injury?
        hosp_er_loc = tbi_1b, # if LOC, how long?
        hosp_er_mem_daze = tbi_1c, # were they dazed or have memory gap?
        hosp_er_age = tbi_1d, # how old were they?
        vehicle_inj = tbi_2, # ever injured in a vehicle accident?
        vehicle_loc = tbi_2b, # if LOC, how long?
        vehicle_mem_daze = tbi_2c, # were they dazed or have memory gap?
```

```
        vehicle_age = tbi_2d, # how old were they?
        fall_hit_inj = tbi_3, # ever injured head/neck from fall or hit?
        fall_hit_loc = tbi_3b, # if LOC, how long?
        fall_hit_mem_daze = tbi_3c, # were they dazed or have memory gap?
        fall_hit_age = tbi_3d, # how old were they?
        violent_hit_inj = tbi_4, # ever injure head/neck from violence?
        violent_hit_loc = tbi_4b, # if LOC, how long?
        violent_hit_mem_daze = tbi_4c, # were they dazed or have memory gap?
        violent_hit_age = tbi_4d, # how old were they?
        blast_inj = tbi_5, # ever injure head or neck from blast?
        blast_loc = tbi_5b, # if LOC, how long?
        blast_mem_daze = tbi_5c, # were they dazed or have memory gap?
        blast_age = tbi_5d, # how old were they?
        other_loc_inj = tbi_6o, # any other injuries with LOC?
        other_loc_num = tbi_6p, # how many more?
        other_loc_max_loc_mins = tbi_6q, # how long was longest LOC?
        other_loc_num_over_30 = tbi_6r, # how many were >= 30 min?
        other_loc_min_age = tbi_6s, # what was their youngest age?
        multi_inj = tbi_7a, # did they have a period of multiple injuries?
        multi_loc = tbi_7c1, # if LOC, how long?
        multi_mem_daze = tbl_7c2, # were they dazed or have memory gap?
        multi_effect_start_age = tbi_7e, # at what age did the effects begin?
        multi_effect_end_age = tbi_7f, # at what age did the effects end?
        other_multi_inj = tbi_7g, # was there another multiple injury period?
        other_multi_effect_type = tbi_7i, # typical effect of the injury?
        other_multi_effect_start_age = tbi_7k, # start age of those effects?
        other_multi_effect_end_age = tbi_7l, # end age of those effects?
        other_other_multi_inj = tbi_8g, # another period of multiple inj?
        other_other_multi_effect_type = tbi_8i, # typical effects?
        other_other_multi_effect_start_age = tbi_8k, # start age of effects?
        other_other_multi_effect_end_age = tbi_8l) # end age of effects?

export_path <- paste0("lab-data/abcd/processed/general/",
    "abcd_otbi01_renamed.csv")
write_csv(abcd_otbi01_renamed, export_path)
```

Note that all variables with the same prefix (or number originally) are referring to the same injury or set of injuries.

6.2 Identifying subjects at a chronic stage post-mTBI

To identify subjects at a chronic stage post-mTBI, two intermediate pieces of information must be calculated from the original dataset:

- identifying which head injuries were mTBIs
- identifying the time since the most recent mTBI

The structure of the Ohio State TBI screen as used for ABCD data collection was not completely compatible with the most commonly used definition of mTBI[3]:

At least one of the following:

- Any period of loss of consciousness
- Any loss of memory
- Any alteration in mental state
- Focal neurological deficits

but where the severity does not exceed:

- LOC over 30 minutes
- A Glasgow Coma Scale score below 13
- Post-traumatic amnesia (PTA) greater than 24 hours

A few aspects of the data collection process in the ABCD study precluded the usage of this definition:

- No information was collected regarding the GCS
- Feeling dazed and PTA were combined into a single variable, making it unclear which combination of symptoms was present
- No information was collected regarding the duration of feeling dazed or experiencing PTA

Consequently, I used the following definition to identify mTBIs:

Any child who experienced at least one injury where that injury resulted in either:

1. Less than 30 minutes of LOC (including 0 minutes) **AND** any duration of memory loss / feeling dazed
2. LOC that is greater than 0 minutes but less than 30 minutes.

The fixed nature of the questionnaire also meant that some information about multiple mTBIs sustained in the same way or about a very large number of mTBIs throughout a child's life could have been lost (see the previous section on identifying children with a history of mTBI). For my analyses, I assume these edge cases are not common enough to meaningfully change any obtained conclusions.

The code below generates a new file which adds the following columns to the TBI screen data:

1. if a subject has experienced an mTBI prior to baseline
2. which of the reported head injuries were mTBIs
3. the time since a subject's most recent mTBI

A few types of injuries were handled differently from the rest in the above code:

- for `other_loc`, the questionnaire asked for a total number of injuries with LOC as well as a total number of injuries with LOC above 30 minutes.
  - If their difference was above 0, they were labeled as having had an mTBI.
- for `other_multi` and `other_other_multi`, insufficient information was provided to determine the severity of the associated injuries.

For my analyses, I consider 3 months to be the threshold at which an mTBI patient enters a chronic stage post-injury. The subject list was then defined in the following code:

6.3 Some participant demographics

6.4 Handling twins

6.5 Identifying matched and non-matched control groups

7 Extracting input variables

The proposed analyses for this project include the following categories of input variables:

- acute symptoms
- brain structure and function
- demographics
- psychosocial factors
- medical history

Over the next few subsections, I go collect key variables related to these categories from the NDA.

7.1 Acute symptoms

Acute symptoms of interest include:

1. mechanism of injury
2. age at which injury occurred
3. presence and duration of LOC
4. presence of feeling dazed or having memory loss following the injury

START OF SCRIPT

Script location:

```
[1] "wheeler-lab/abcd/prashanth-proj/03-get-inputs/get_acute_symptoms.Rmd"
```

```
library(tidyverse)
library(magrittr)

abcd_otbi01_detailed <- read_delim("lab-data/abcd/processed/prashanth-proj/abcd_otbi01_det
subject_file <- paste0("lab-data/abcd/processed/prashanth-proj/",
                       "baseline-chronic-mtbi/subjects.csv")
subjects <- read_delim(subject_file, delim=",")
acute_symptoms <- merge(abcd_otbi01_detailed, subjects)
```

```r
# 1. Identify mechanism of injury
mpis <- acute_symptoms %>%
    select(matches("subjectkey") | ends_with("mtbi_mpi"))

latest_mtbi_mechanism <- colnames(mpis)[apply(mpis,1,which.max)] %>%
{gsub("_mtbi_mpi", "", .)}

acute_symptoms$latest_mtbi_mechanism <- latest_mtbi_mechanism


# 2. Identify age of injury
acute_symptoms %<>%
    mutate(latest_mtbi_age = case_when(
        latest_mtbi_mechanism == "hosp_er" ~ hosp_er_age,
        latest_mtbi_mechanism == "vehicle" ~ vehicle_age,
        latest_mtbi_mechanism == "fall_hit" ~ fall_hit_age,
        latest_mtbi_mechanism == "violent_hit" ~ violent_hit_age,
        latest_mtbi_mechanism == "blast" ~ blast_age,
        latest_mtbi_mechanism == "other_loc" ~ other_loc_min_age,
        latest_mtbi_mechanism == "multi" ~ multi_effect_end_age))


# 3. Identify presence & duration of LOC
## 3-1 first create a compatible LOC variable for the 'other_mtbi' category
acute_symptoms %<>%
    mutate(other_loc_loc = case_when(
        other_loc_max_loc_mins == 0 ~ 0,
        other_loc_max_loc_mins > 0 & other_loc_max_loc_mins <= 30 ~ 1,
        other_loc_max_loc_mins > 30 & other_loc_max_loc_mins <= 1440 ~ 2,
        other_loc_max_loc_mins > 1440 ~ 3,
        TRUE ~ NA_real_))

acute_symptoms %<>%
    mutate(latest_mtbi_loc = case_when(
        latest_mtbi_mechanism == "hosp_er" ~ hosp_er_loc,
        latest_mtbi_mechanism == "vehicle" ~ vehicle_loc,
        latest_mtbi_mechanism == "fall_hit" ~ fall_hit_loc,
        latest_mtbi_mechanism == "violent_hit" ~ violent_hit_loc,
        latest_mtbi_mechanism == "blast" ~ blast_loc,
        latest_mtbi_mechanism == "other_loc" ~ other_loc_loc,
```

```
        latest_mtbi_mechanism == "multi" ~ multi_loc,
        TRUE ~ NA_real_))

# 4. Identify presence of feeling dazed / memory loss following injury
acute_symptoms %<>%
    mutate(latest_mtbi_mem_daze = case_when(
        latest_mtbi_mechanism == "hosp_er" ~ hosp_er_mem_daze,
        latest_mtbi_mechanism == "vehicle" ~ vehicle_mem_daze,
        latest_mtbi_mechanism == "fall_hit" ~ fall_hit_mem_daze,
        latest_mtbi_mechanism == "violent_hit" ~ violent_hit_mem_daze,
        latest_mtbi_mechanism == "blast" ~ blast_mem_daze,
        latest_mtbi_mechanism == "other_loc" ~ NA_real_,
        latest_mtbi_mechanism == "multi" ~ multi_mem_daze,
        TRUE ~ NA_real_))

# 5. Trim acute_symptoms to only the required variables
acute_symptoms %<>%
    select(subjectkey,
           latest_mtbi_mechanism,
           latest_mtbi_age,
           latest_mtbi_loc,
           latest_mtbi_mem_daze)

output_path <- paste0("lab-data/abcd/processed/prashanth-proj/",
    "baseline-chronic-mtbi/inputs/acute_symptoms.csv")
write_csv(acute_symptoms, output_path)
```

7.2 Brain structure and function

Variables of interest related to brain structure and function include:

- Structural: T1-weighted multishell diffusion-weighted MRI data
- Functional: resting-state functional MRI data

The 2019 publication by Hagler et al.[4] outlines the neuroimaging data present in the ABCD dataset in detail.

7.2.1 Brain structure

DTI measures are provided for subcortical regions, cortical regions, and major white matter fiber tracts. The analyses in this report are only concerned with white matter structures, which are present in the following ROIs:

1. Fiber tracts automatically labeled by AtlasTrack [Hagler] (supplementary table 6 of Hagler 2019 file 1)

E.g.:

- Element: dmdtifp1_1
- Name: Average fractional anisotropy within DTI atlas tract right fornix
- Aliases: dmri_dti.full.fa_fiber.at_fx.rh, dmri_dtifullfa_fiberat_fxrh

2. Subcortical regions labeled by automated segmentation [Fischl] (supplementary table 1 of Hagler 2019 file 1)

E.g.:

- Element: dmdtifp1_211
- Name: Average fractional anisotropy within ASEG ROI left-cerebral-white-matter
- Aliases: dmri_dti.full.md_subcort.aseg_cerebellum.white.matter.lh, dmri_dtifullmd_subcortaseg_cereb

3. Regions of white matter sub-adjacent to cortical regions defined by anatomical parcellation [Desikan] (supplementary table 2 of Hagler 2019 file 1)

E.g.:

- Element: dmdtifp1_343
- Name: Average fractional anisotropy within parcellation of sub-adjacent white matter associated with cortical ROI left-medial orbito frontal
- Aliases: dmri_dti.full.fa.wm_cort.desikan_medialorbitofrontal.lh, dmri_dtifullfawm_cortdesikan_media

Detailed lists of the ROIs are present in Supplementary File 2 of[4].

- Average fractional anisotropy (FA)
- Mean diffusivity (MD)
- Average longitudinal diffusion coefficient
- Average transverse diffusion coefficient
- Fiber tract volume

For my current analyses, I include:

- non-compound regions
- FA and MD

(include rationale for why only look at these 2 and why not include GM)

The dTI data are located in the files *abcd_dmdtifp101.txt* and *abcd_dmdtifp201* (titled ABCD dMRI DTI Full Part 1 and 2 respectively). All variables follow the naming scheme 'dmdtifp1_x', where x ranges from 1 to 1185.

Some dMRI variables for white matter regions are shown below:

Variable	Segmentation method	Region type	Element name in the NDA
Fractional anisotropy	AtlasTrack	Major WM tracts	dmdtifp1_1 to dmdtifp1_35
Fractional anisotropy	Anatomical parcellation	Sub-adjacent white matter	dmdtifp1_331 to dmdtifp1_401
Fractional anisotropy	Automated segmentation	WM of major brain region	dmdtifp1_211, 214, 228, 231
Mean diffusivity	AtlasTrack	Major WM tracts	dmdtifp1_43 to dmdtifp1_79
Mean diffusivity	Anatomical parcellation	Sub-adjacent white matter	dmdtifp1_402 to dmdtifp1_472
Mean diffusivity	Automated segmentation	WM of major brain region	dmdtifp1_241, 244, 258, 261

I look at the parcellation used by Desikan et al., which covers dmtifp_331 to 472

4. presence of feeling dazed or having memory loss following the injury

7.3 Brain structure and function

7.4 Demographics (!)

!Known issues

- I am unsure of how clustering over categorical (nominal) variables, such as race or sex, will influence outcomes
- The inclusion of these variables will simply incentivize the clustering algorithm to assign members from the same sex/race/ethnicity to the same cluster
- May be worth it to try clustering with and without these variables and check how outcomes differ

Demographic variables to be included in the analyses are:

- Age
- Sex
- Race
- Ethnicity
- Socioeconomic status
- Pubertal development status

Age and sex are commonly included in the ABCD study's data files.

7.5 Psychosocial factors

Psychosocial factors of interest include:

- family function
- prosocial behaviour
- loneliness
- healthy behaviours
- parent psychopathology

7.6 Medical history

Medical history variables of interest include:

- Number of previous head injuries
- History of headaches

7.7 Handling data types of mixed input

The original similarity network fusion paper[1] suggests using a scaled Euclidean distance for continuous variables, chi-squared distance for discrete variables, and agreement-based measure for binary variables. The paper also indicates that SNF may be used to incorporate arbitrary types of discrete (binary or categorical) data alongside continuous data, and that the compatibility of the selected data sources can be verified using normalized mutual information scores. This topic is addressed further in the methods section.

7.8 Post-concussion symptoms

8 Similarity network fusion

In this chapter I run through the SNF pipeline, which consists of:

1. Generating similarity matrices
2. Applying SNF
3. Clustering the SNF results into individual subtypes

START OF SCRIPT

Script location:

```
[1] "wheeler-lab/abcd/prashanth-proj/04-snf/get_demographics.Rmd"
```

```
library(SNFtool)
library(ggsignif)
library(data.table)
library(magrittr)
library(tidyverse)
library(fastDummies)
library(ggplot2)


# 1. Preparing data for SNF ================================================
data_path <- paste0("lab-data/abcd/processed/prashanth-proj/",
                    "baseline-chronic-mtbi/inputs/")


## 1-1 Acute symptom information-------------------------------------------
acute_symptoms <- read.csv(paste0(data_path, "acute_symptoms.csv"))


### 1-1a Mechanism of injury
mechanism_pre_dummy <- acute_symptoms %>%
  select("subjectkey", "latest_mtbi_mechanism")

mechanism <- dummy_cols(mechanism_pre_dummy,
```

```
                        select_columns = "latest_mtbi_mechanism",
                        remove_selected_columns = TRUE)
rm(mechanism_pre_dummy)


### 1-1b Age at injury
injury_age <- acute_symptoms %>%
  select("subjectkey", "latest_mtbi_age")


### 1-1c Effects (LOC & "PTA") of mTBI
mtbi_effects <- acute_symptoms %>%
  select("subjectkey", "latest_mtbi_loc", "latest_mtbi_mem_daze")
rm(acute_symptoms)


## 1-2 Brain structure-----------------------------------------------------
brain_str <- read.csv(paste0(data_path, "brain_str.csv"))

### 1-2a Fractional anisotropy
fa <- brain_str %>%
    select("subjectkey", "dmdtifp1_331":"dmdtifp1_401")

### 1-2b Mean diffusivity
md <- brain_str %>%
    select("subjectkey", "dmdtifp1_402":"dmdtifp1_472")
rm(brain_str)


### Demographics
demographics <- read.csv(paste0(data_path, "demographics.csv"))

# 1. ses
ses <- demographics %>%
  select("subjectkey", "acs_raked_propensity_score")

# 2. age
age <- demographics %>%
  select("subjectkey", "interview_age")

# 3. sex
```

```r
sex_pre_dummy <- demographics %>%
  select("subjectkey", "sex")

sex <- dummy_cols(sex_pre_dummy, select_columns = "sex", remove_selected_columns = TRUE)

rm(sex_pre_dummy)

# 4. puberty
puberty <- demographics %>%
  select("subjectkey", "sex", starts_with("pds"))

puberty$puberty_m <- rowMeans(puberty[, c("pds_p_ss_male_category",
                                          "pds_y_ss_male_category")],
                              na.rm=TRUE)
puberty$puberty_f <- rowMeans(puberty[, c("pds_p_ss_female_category",
                                          "pds_y_ss_female_category")],
                              na.rm=TRUE)

puberty %<>%
  select("subjectkey", starts_with("puberty"))

puberty$puberty <- rowSums(puberty[, c("puberty_m",
                                       "puberty_f")],
                           na.rm=TRUE)

puberty %<>%
  select("subjectkey", "puberty")

# 5. race
race <- demographics %>%
  select("subjectkey", "white":"hispanic")

rm(demographics)

### Psychosocial factors


### Medical history

# My variables:
quick <- function(dataframe) {
```

```
    if (ncol(dataframe) >= 5) {
    return(head(dataframe[,1:5]))
    } else {
    return(head(dataframe))
    }
}

race <- arrange(race, subjectkey)
ses <- arrange(ses, subjectkey)
sex <- arrange(sex, subjectkey)
puberty <- arrange(puberty, subjectkey)
mechanism <- arrange(mechanism, subjectkey)
mtbi_effects <- arrange(mtbi_effects, subjectkey)
injury_age <- arrange(injury_age, subjectkey)
fa <- arrange(fa, subjectkey)
md <- arrange(md, subjectkey)
age <- arrange(age, subjectkey)

quick(race)
quick(age)
quick(fa)
quick(md)
quick(injury_age)
quick(mechanism)
quick(mtbi_effects)
quick(puberty)
quick(ses)
quick(sex)
# Looks good


na_removal <- function(dataframe) {
    start_row <- nrow(dataframe)
    start_col <- ncol(dataframe)
    na_status <- any(is.na(dataframe))
    dataframe <- na.omit(dataframe)
    end_row <- nrow(dataframe)
    end_col <- ncol(dataframe)
    print(paste0("The presence of NAs in this object is ",
                 na_status, ".",
                 " Converted a ",start_row,"x", start_col,
```

```
                " dataframe to ",end_row, "x", end_col))
    return(dataframe)
}

age <- na_removal(age)
fa <- na_removal(fa)
md <- na_removal(md)
injury_age <- na_removal(injury_age)
mechanism <- na_removal(mechanism)
mtbi_effects <- na_removal(mtbi_effects)
puberty <- na_removal(puberty)
race <- na_removal(race)
ses <- na_removal(ses)
sex <- na_removal(sex)


common_subs <- Reduce(intersect, list(age$subjectkey,
                                      fa$subjectkey,
                                      md$subjectkey,
                                      injury_age$subjectkey,
                                      mechanism$subjectkey,
                                      mtbi_effects$subjectkey,
                                      puberty$subjectkey,
                                      race$subjectkey,
                                      ses$subjectkey,
                                      sex$subjectkey))


age <- age[age$subjectkey %in% common_subs, ]
fa <- fa[fa$subjectkey %in% common_subs, ]
md <- md[md$subjectkey %in% common_subs, ]
injury_age <- injury_age[injury_age$subjectkey %in% common_subs, ]
mechanism <- mechanism[mechanism$subjectkey %in% common_subs, ]
mtbi_effects <- mtbi_effects[mtbi_effects$subjectkey %in% common_subs, ]
puberty <- puberty[puberty$subjectkey %in% common_subs, ]
race <- race[race$subjectkey %in% common_subs, ]
ses <- ses[ses$subjectkey %in% common_subs, ]
sex <- sex[sex$subjectkey %in% common_subs, ]


prod(age$subjectkey == fa$subjectkey)
```

```r
prod(fa$subjectkey == md$subjectkey)
prod(md$subjectkey == injury_age$subjectkey)
prod(injury_age$subjectkey == mechanism$subjectkey)
prod(mechanism$subjectkey == mtbi_effects$subjectkey)
prod(mtbi_effects$subjectkey == puberty$subjectkey)
prod(puberty$subjectkey == race$subjectkey)
prod(race$subjectkey == ses$subjectkey)
prod(ses$subjectkey == sex$subjectkey)




age <- data.frame(age, row.names = 1)
fa <- data.frame(fa, row.names = 1)
md <- data.frame(md, row.names = 1)
injury_age <- data.frame(injury_age, row.names = 1)
mechanism <- data.frame(mechanism, row.names = 1)
mtbi_effects <- data.frame(mtbi_effects, row.names = 1)
puberty <- data.frame(puberty, row.names = 1)
race <- data.frame(race, row.names = 1)
ses <- data.frame(ses, row.names = 1)
sex <- data.frame(sex, row.names = 1)


# convert matrices to numeric
age <- as.matrix(data.frame(lapply(age, function(x) as.numeric(as.character(x))), check.na
fa <- as.matrix(data.frame(lapply(fa, function(x) as.numeric(as.character(x))), check.name
md <- as.matrix(data.frame(lapply(md, function(x) as.numeric(as.character(x))), check.name
injury_age <- as.matrix(data.frame(lapply(injury_age, function(x) as.numeric(as.character(
mechanism <- as.matrix(data.frame(lapply(mechanism, function(x) as.numeric(as.character(x)
mtbi_effects <- as.matrix(data.frame(lapply(mtbi_effects, function(x) as.numeric(as.charac
puberty <- as.matrix(data.frame(lapply(puberty, function(x) as.numeric(as.character(x))),
race <- as.matrix(data.frame(lapply(race, function(x) as.numeric(as.character(x))), check.
ses <- as.matrix(data.frame(lapply(ses, function(x) as.numeric(as.character(x))), check.na
sex <- as.matrix(data.frame(lapply(sex, function(x) as.numeric(as.character(x))), check.na

# snftool tutorial -------------------------------------------------------
# https://rdrr.io/cran/SNFtool/man/SNF.html

# initial parameters
```

```r
k <- 39             # number of nearest neighbors - suggested as n/10
alpha <- 0.5        # hyperparameter
iterations <- 20


# euclidean distance
ed <- function(dataframe) {
    return(as.matrix(dist(dataframe, method="euclidean")))
}

bd <- function(dataframe) {
    return(as.matrix(dist(dataframe, method="binary")))
}



# construct distance matrices
age_distances <- ed(age)
fa_distances <- ed(fa)
md_distances <- ed(md)
injury_age_distances <- ed(injury_age)
puberty_distances <- ed(puberty)
ses_distances <- ed(ses)
race_distances <- bd(race)
mechanism_distances <- bd(mechanism)
mtbi_effects_distances <- bd(mtbi_effects)
sex_distances <- bd(sex)


# construct similarity graphs (W matrices)
age_similarity_graph <- affinityMatrix(age_distances, k, alpha)
fa_similarity_graph <- affinityMatrix(fa_distances, k, alpha)
md_similarity_graph <- affinityMatrix(md_distances, k, alpha)
injury_age_similarity_graph <- affinityMatrix(injury_age_distances, k, alpha)
mechanism_similarity_graph <- affinityMatrix(mechanism_distances, k, alpha)
mtbi_effects_similarity_graph <- affinityMatrix(mtbi_effects_distances, k, alpha)
puberty_similarity_graph <- affinityMatrix(puberty_distances, k, alpha)
race_similarity_graph <- affinityMatrix(race_distances, k, alpha)
ses_similarity_graph <- affinityMatrix(ses_distances, k, alpha)
sex_similarity_graph <- affinityMatrix(sex_distances, k, alpha)
```

```
fused_graph <- SNF(list(age_similarity_graph,
                        fa_similarity_graph,
                        md_similarity_graph,
                        injury_age_similarity_graph,
                        mechanism_similarity_graph,
                        mtbi_effects_similarity_graph,
                        puberty_similarity_graph,
                        race_similarity_graph,
                        ses_similarity_graph,
                        sex_similarity_graph))

# rankFeaturesByNMI(list(age_similarity_graph,
#                        fa_similarity_graph,
#                        md_similarity_graph,
#                        injury_age_similarity_graph,
#                        mechanism_similarity_graph,
#                        mtbi_effects_similarity_graph,
#                        puberty_similarity_graph,
#                        race_similarity_graph,
#                        ses_similarity_graph,
#                        sex_similarity_graph), fused_graph)


# DEBUG NMI
#rankFeaturesByNMI <- function(data, W){
#    # Calculates the normalized mutual information (NMI) score between each
#    # features clustering and the clustering of the fused matrix W. Each feature
#    # is ranked based on how similar it is to the clustering of the fused matrix.
#    #
#    # Args:
#    #   data: A list of matrices
#    #   W: Fused affinity matrix from all data types in data
#    #
#    # Returns:
#    #   A list containing NMI score for each feature from all data types
#    #   and their NMI score ranks.
#    #
#
#    stopifnot(class(data) == "list")
#
#    NUM.OF.DATA.TYPES <- length(data)
```

```
#     NMI.scores <- vector(mode="list", length=NUM.OF.DATA.TYPES)
#     NMI.ranks <- vector(mode="list", length=NUM.OF.DATA.TYPES)
#     num.of.clusters.fused <- estimateNumberOfClustersGivenGraph(W)[[1]]
#     clustering.fused <- spectralClustering(W, num.of.clusters.fused)
#
#     for (data.type.ind in 1:NUM.OF.DATA.TYPES){
#         NUM.OF.FEATURES <- dim(data[[data.type.ind]])[2]
#         NMI.scores[[data.type.ind]] <- vector(mode="numeric",
#             length=NUM.OF.FEATURES)
#
#         for (feature.ind in 1:NUM.OF.FEATURES){
#             affinity.matrix <- affinityMatrix(
#                 dist2(as.matrix(data[[data.type.ind]][, feature.ind]),
#                 as.matrix(data[[data.type.ind]][, feature.ind])))
#
#             clustering.single.feature <- spectralClustering(affinity.matrix,
#                 num.of.clusters.fused)
#
#             NMI.scores[[data.type.ind]][feature.ind] <- calNMI(clustering.fused,
#                 clustering.single.feature)
#         }
#
#         NMI.ranks[[data.type.ind]] <- rank(-NMI.scores[[data.type.ind]],
#              ties.method="first")
#     }
#
#     return(list(NMI.scores, NMI.ranks))
#}
```

```r
estimateNumberOfClustersGivenGraph(fused_graph, 2:10)
```

```r
# Try this with 2 subtypes next time
C <- 3
group <- spectralClustering(fused_graph, C)
displayClusters(fused_graph, group)
displayClustersWithHeatmap(fused_graph, group)

plot(age, col=group, main='age')
plot(fa, col=group, main='fa')


subtype_list <- data.frame(common_subs, group)
# write_csv(subtype_list,
#           "lab-data/abcd/processed/prashanth-proj/baseline-chronic-mtbi/subtype_list.csv"

# Plots
# 1. Prepare input variables for plotting
df_sublabel <- function(d, group=parent.frame()$group) {
    d <- as.data.frame(d)
    d$subtype <- group
    d$subtype <- as.factor(d$subtype)
    return(d)
}

age_post <- df_sublabel(age)
fa_post <- df_sublabel(fa)
injury_age_post <- df_sublabel(injury_age)
md_post <- df_sublabel(md)
mechanism_post <- df_sublabel(mechanism)
mtbi_effects_post <- df_sublabel(mtbi_effects)
puberty_post <- df_sublabel(puberty)
race_post <- df_sublabel(race)
```

```r
ses_post <- df_sublabel(ses)
sex_post <- df_sublabel(sex)


# 2. Generate functions for plotting continuous and categorical data

## 2-1 Jitter plots for continuous variables
cont_jitter <- function(d, y, ylabel, top_sig = max(d[, y])*1.02, sigsize = 16, ymin = min
    jitter_plot <- ggplot(data = d, aes_string(x = "subtype", y = y)) +
                   geom_boxplot() +
                   geom_jitter(size=3, width=0.1, alpha=0.5, aes_string(color = "subtype")
                   coord_cartesian(ylim=c(ymin, ymax)) +
                   theme_classic() +
                   theme(text = element_text(size = 40), legend.position="none") +
                   geom_signif(comparisons = list(c("1", "2")),
                               color = "black",
                               textsize = sigsize,
                               map_signif_level = TRUE) +
                   geom_signif(comparisons = list(c("2", "3")),
                               color = "black",
                               textsize = sigsize,
                               map_signif_level = TRUE) +
                   geom_signif(comparisons = list(c("1", "3")),
                               color="black",
                               y_position = top_sig,
                               textsize = sigsize,
                               map_signif_level = TRUE) +
                   xlab("Subtype") +
                   ylab(ylabel)
    return(jitter_plot)
}


cont_jitter2 <- function(d, y, ylabel, top_sig = max(d[, y])*1.02, sigsize = 20, ymin = mi
    jitter_plot <- ggplot(data = d, aes_string(x = "subtype", y = y)) +
                   geom_boxplot() +
                   geom_jitter(size=3, width=0.1, alpha=0.5, aes_string(color = "subtype")
                   coord_cartesian(ylim=c(ymin, ymax)) +
                   theme_classic() +
                   theme(text = element_text(size = 25)) +
                   geom_signif(comparisons = list(c("1", "2")),
```

```
                                color = "black",
                                textsize = sigsize,
                                map_signif_level = TRUE,
                                test = "t.test",
                                test.args = list(var.equal = TRUE)) +
                    xlab("Subtype") +
                    ylab(ylabel)
    return(jitter_plot)
}



## 2-2 Bar charts for continuous variables - WIP
cont_bar <- function(d, y, ymin=0, ymax=100, ylabel) {
    agg_data <- aggregate(get(y, d),
    by = list(subtype = get("subtype", d)),
    FUN = function(x) c(mean = mean(x), sd = sd(x), n = length(x)))
    agg_data <- do.call(data.frame, agg_data)
    agg_data$se <- agg_data$x.sd / sqrt(agg_data$x.n)
    colnames(agg_data) <- c("subtype", "mean", "sd", "n", "se")
    dodge <- position_dodge(width = 0.9)
    limits <- aes(ymax = mean + se,
                  ymin = mean - se)
    bar_plot  <- ggplot(data = agg_data, aes(x = subtype, y = mean, fill = subtype)) +
                 geom_bar(stat = "identity", position = dodge) +
                 geom_errorbar(limits, position = dodge, width = 0.25) +
                 coord_cartesian(ylim=c(ymin, ymax)) +
                 theme_classic() +
                 theme(text = element_text(size = 25)) +
                 xlab("Subtype") +
                 ylab(ylabel)
    plot_dir <- paste0("~/mnt/wheeler-lab/lab-data/abcd/processed/",
                       "prashanth-proj/baseline-chronic-mtbi/figures/")
    plot_name <- paste0(y, "_bar_plot.png")
    return(bar_plot)
}


## 2-3 Pie charts for categorical variables
pie_plot <- function(d, y, column_names, group, colour, fontsize) {
```

```
    tab <- d %>%
        group_by(as.character(d[, y])) %>%
        summarize(n())
    tab <- as.data.frame(tab)
    colnames(tab) <- column_names
    pie(tab$counts,
        labels=as.character(d[, y]),
        col=colour,
        cex=fontsize)
}

## 2-4 Bar charts for categorical variables
cat_bar_classic <- function(d, y) {
    g <- ggplot(d, aes_string(x = "subtype", fill = "sex")) +
         geom_bar(position = "fill") +
         ylab("Proportion") +
         xlab("Subtype") +
         theme_classic() +
         theme(text = element_text(size = 25))
    return(g)
}


cat_bar <- function(d, y, scheme="Spectral") {
    g <- ggplot(d, aes_string(x = "subtype", fill = y)) +
         geom_bar(position = "fill") +
         ylab("Proportion") +
         xlab("Subtype") +
         theme_classic() +
         theme(text = element_text(size = 40)) +
         scale_fill_brewer(palette=scheme)
    return(g)
}

cat_bar2 <- function(d, y) {
    a <- group_by(d, subtype)
    b <- count(a, !!sym(y))
    c <- mutate(b, proportion = n/sum(n))
    ggplot(c, aes_string("subtype", "proportion", fill = y)) +
    geom_col(position = position_stack()) +
    geom_text(aes(label = n), position = position_stack(vjust =0.5), size =12) +
```

```
    theme_classic() +
    theme(text = element_text(size = 50)) +
    ylab("Proportion") +
    xlab("Subtype")
}


cat_bar3 <- function(d, y, scheme="Dark2") {
    a <- group_by(d, subtype)
    b <- count(a, !!sym(y))
    c <- mutate(b, proportion = n/sum(n))
    ggplot(c, aes_string("subtype", "proportion", fill = y)) +
    geom_col(position = position_stack()) +
    geom_text(aes(label = n), position = position_stack(vjust =0.5), size =12) +
    theme_classic() +
    theme(text = element_text(size = 40)) +
    ylab("Proportion") +
    xlab("Subtype") +
    scale_fill_brewer(palette = scheme)
}




## Save-plot function
plotsave <- function(plotname) {
    plot_dir <- paste0("~/mnt/wheeler-lab/lab-data/abcd/processed/",
                       "prashanth-proj/baseline-chronic-mtbi/figures/")
    ggsave(paste0(plot_dir, plotname))
}

plotsave2 <- function(plotname) {
    plot_dir <- paste0("~/mnt/wheeler-lab/lab-data/abcd/processed/",
                       "prashanth-proj/baseline-chronic-mtbi/figures/")
    ggsave(paste0(plot_dir, plotname), height=10, width=10)
}

plotsave3 <- function(plotname) {
    plot_dir <- paste0("~/mnt/wheeler-lab/lab-data/abcd/processed/",
                       "prashanth-proj/baseline-chronic-mtbi/figures/")
    ggsave(paste0(plot_dir, plotname), height=7, width=10)
```

```
}

plotsave4 <- function(plotname, heights=12) {
    plot_dir <- paste0("~/mnt/wheeler-lab/lab-data/abcd/processed/",
                       "prashanth-proj/baseline-chronic-mtbi/figures/")
    ggsave(paste0(plot_dir, plotname), height=heights, width=10)
}



# 3. Making the plots for input variables

# 3-1 age on interview
agey_post <- age_post
agey_post$interview_age <- agey_post$interview_age / 12

cont_jitter(d = agey_post,
            y = "interview_age",
            ylabel = "Age at data collection (y)",
            top_sig = 11.3)
#plotsave("input_agey_jitter_bigfont.png")


ggplot(agey_post, aes(interview_age, fill = subtype, color = subtype)) +
    geom_density(show.legend = T, alpha = 0.2, aes(y = ..count..)) +
    theme_classic() +
    theme(text = element_text(size = 20)) +
    xlab("Age at data collection (y)") +
    ylab("Count") +
    coord_cartesian(xlim = c(min(agey_post$interview_age), max(agey_post$interview_age)),
# plotsave("input_agey_density.png")


# 3-2 age on injury
injury_agey_post <- injury_age_post
injury_agey_post$latest_mtbi_age <- injury_agey_post$latest_mtbi_age / 12

cont_jitter(d = injury_agey_post,
            y = "latest_mtbi_age",
            ylabel = "Age at injury",
            ymax = 12,
```

```
            top_sig = 11.3)
plotsave4("input_injury_agey_jitter_bigfont3.png")
# Looks strange because age at injury was provided in years


# 3-3 ses
cont_jitter(d = ses_post,
            y = "acs_raked_propensity_score",
            ylabel = "Socioeconomic status score",
            ymax = 2000,
            top_sig = 1900)
plotsave4("input_ses_jitter_bigfont4.png")

## 3-4 sex
sex_post %<>%
    mutate(sex = case_when(sex_F == 1 ~ "F", TRUE ~ "M")) %>%
    select("sex", "subtype")
sex_post$sex <- as.factor(sex_post$sex)

cat_bar(sex_post, "sex", scheme="Dark2")
plotsave2("input_sex_bar_numbered6.png")

pie_plot(d = sex_post,
         y = "sex",
         column_names = c("sex", "counts"),
         colour = c("red", "blue"),
         fontsize = 2)
# plotsave("input_sex_pie.png")


## 3-5 puberty
min(puberty_post$puberty)
puberty_post <- puberty_post[which(puberty_post$puberty != 0), ]
dim(puberty_post)

cont_jitter(d = puberty_post,
            y = "puberty",
            ylabel = "Pubertal Development Scale Score",
            ymin = 0.5,
            ymax = 4.5,
            top_sig = 4.3)
```

```r
# plotsave("input_puberty_jitter.png")

## 4
mechanism_undummied <- mechanism_post %>%
    mutate(mechanism = case_when(latest_mtbi_mechanism_fall_hit == 1 ~ "fall/hit",
                      latest_mtbi_mechanism_hosp_er == 1 ~ "hospitalized",
                      latest_mtbi_mechanism_multi == 1 ~ "multiple",
                      latest_mtbi_mechanism_other_loc == 1 ~ "other_loc",
                      latest_mtbi_mechanism_vehicle == 1 ~ "vehicle",
                      latest_mtbi_mechanism_violent_hit == 1 ~ "violent"))


cat_bar(mechanism_undummied, "mechanism", scheme = "Dark2" )
# plotsave("input_mechanism_bar.png")

mtbi_effects_post %<>%
    mutate(LOC = case_when(latest_mtbi_loc == 0 ~ "No",
                           TRUE ~ "Yes"))

mtbi_effects_post %<>%
    mutate(mem_daze = case_when(latest_mtbi_mem_daze == 0 ~ "No",
                           TRUE ~ "Yes"))

mtbi_effects_post %<>%
    rename(`Memory loss or dazed` = "mem_daze")

cat_bar(mtbi_effects_post, "LOC", scheme = "Dark2")
plotsave2("input_loc2.png")
# plotsave("input_loc.png")

cat_bar(mtbi_effects_post, "mem_daze", scheme = "Dark2")
plotsave2("input_mem_daze2.png")
#plotsave("input_mem_daze.png")

colnames(race_post)

race_post_merged <- race_post

race_post_merged %<>%
    mutate(mixed = case_when(white +
                             black +
```

```
                                native_american +
                                native_alaskan +
                                native_hawaiian +
                                guamanian +
                                samoan +
                                other_pacific_islander +
                                asian +
                                chinese +
                                filipino +
                                japanese +
                                korean +
                                vietnamese +
                                other_asian +
                                other +
                                hispanic > 1 ~ 1,
                            TRUE ~ 0))

race_post_merged %<>%
    mutate(race = case_when(mixed == 1 ~ "Mixed",
                            white == 1 ~ "White",
                            black == 1 ~ "Black",
                            native_american == 1 ~ "Native American",
                            native_alaskan == 1 ~ "Native Alaskan",
                            native_hawaiian == 1 ~ "Native Hawaiian",
                            guamanian == 1 ~ "Guamanian",
                            samoan == 1 ~ "Samoan",
                            other_pacific_islander == 1 ~ "Other Pacific Islander",
                            chinese == 1 ~ "Chinese",
                            filipino == 1 ~ "Filipino",
                            japanese == 1 ~ "Japanese",
                            korean == 1 ~ "Korean",
                            vietnamese == 1 ~ "Vietnamese",
                            asian == 1 ~ "Other Asian",
                            other_asian == 1 ~ "Other Asian",
                            other == 1 ~ "Other or NA",
                            refuse_to_answer == 1 ~ "Other or NA",
                            hispanic == 1 ~ "Hispanic"))
race_post_merged %<>%
    rename(Race = race)

cat_bar(race_post_merged, "Race", scheme="Dark2")
```

```r
plotsave2("input_race_bar2.png")
# plotsave("input_race_bar.png")




# Quick section to describe the inputs
df_list <- list(age_post,
                injury_age_post,
                mechanism_undummied,
                mtbi_effects_post,
                puberty_post,
                race_post_merged,
                ses_post,
                sex_post)

lapply(df_list, quick)

df_list_named <- lapply(df_list, function(x) setDT(x, keep.rownames = TRUE)[])
full_inputs <- df_list_named %>% reduce(full_join, by=c('rn', 'subtype'))

library(tools)

full_inputs <- rename_with(full_inputs, ~ toTitleCase(gsub("_", " ", .x, fixed = TRUE)))
full_inputs <- rename_with(full_inputs, ~ gsub("Acs", "ACS", .x, fixed = TRUE))
full_inputs <- rename_with(full_inputs, ~ gsub("Mtbi", "mTBI", .x, fixed = TRUE))
full_inputs <- rename_with(full_inputs, ~ gsub("Loc", "LOC", .x, fixed = TRUE))

full_inputs_renamed <- full_inputs %>%
    mutate(Race = case_when(
        Race == "white" ~ "White",
        Race == "mixed" ~ "Mixed",
        Race == "black" ~ "Black",
        Race == "other/refuse_to_answer" ~ "Other or NA",
        Race == "hispanic" ~ "Hispanic",
        Race == "other_asian" ~ "Other or NA",
        Race == "chinese" ~ "Asian",
        Race == "filipino" ~ "Asian")) %>%
    mutate(Mechanism = case_when(
```

```
        Mechanism == "fall/hit" ~ "Fall",
        Mechanism == "hospitalized" ~ "ER visit",
        Mechanism == "multiple" ~ "Other",
        Mechanism == "vehicle" ~ "Vehicle accident",
        Mechanism == "violent" ~ "Other")) %>%
    rename(`Memory loss or feeling dazed` = `Mem Daze`)


library(table1)
table1(~`Mechanism` +
       `Race`, data = full_inputs_renamed)
table1(~`Memory loss or feeling dazed` +
       `LOC` +
       `Latest mTBI Age` +
       `Sex`, data = full_inputs_renamed)

table1(~Sex, data = full_inputs)


# 4 SNF graph
library("qgraph")

subtype1 <- which(group == 1)
subtype2 <- which(group == 2)
subtype3 <- which(group == 3)

graphsize <- 390
minigraph <- fused_graph[1:graphsize, 1:graphsize]
minigroup <- group[1:graphsize]

subtype1 <- which(minigroup == 1)
subtype2 <- which(minigroup == 2)
subtype3 <- which(minigroup == 3)

matrix_vals <- as.vector(minigraph)
hist(matrix_vals)



# matrix_vals_minimal <- matrix_vals[which(matrix_vals < 0.5)]
# hist(matrix_vals_minimal)
```

```r
# quantile(matrix_vals_minimal, probs = c(.95, .99, .999))

plot_dir <- paste0("~/mnt/wheeler-lab/lab-data/abcd/processed/",
                   "prashanth-proj/baseline-chronic-mtbi/figures/")

qgraph(minigraph, layout="spring",
       labels = FALSE,
       groups = list("Subtype 1" = subtype1,
                     "Subtype 2" = subtype2,
                     "Subtype 3" = subtype3),
                    color = c("salmon",
                              "lightgreen",
                              "lightblue"),
       edge.color = "darkgray",
       minimum = 0.003,
       vsize = 1.5,
       filename = paste0(plot_dir,"qgraph_full"),
       filetype = "png")

qgraph(minigraph,
       labels=FALSE,
       groups = list("Subtype 1" = subtype1,
                     "Subtype 2" = subtype2,
                     "Subtype 3" = subtype3),
       color = c("salmon",
                 "lightgreen",
                 "lightblue"),
       edge.color = "darkgray",
       vsize = 3)
hist(minigraph)


# next step: eliminate the low-weight edges

qgraph(minigraph, layout="groups",
       labels=FALSE,
       groups = list("Subtype 1" = subtype1,
                     "Subtype 2" = subtype2,
                     "Subtype 3" = subtype3),
       graph = "factorial")
```

```
qgraph(minigraph, graph="factorial")

qgraph(a, layout="spring",
       groups = list(Depression = 1:16,
                     "OCD" = 17:26),
                     color = c("lightblue",
                     "lightsalmon"))

print(5+5)
```

9 Next steps

Here is where I will talk about what I plan on doing next.

10 Limitations

Here is where I discuss the limitations of my project.

11 Wrapping-up

Discussion

Conclusion

References

1. Wang, B. *et al.* Similarity network fusion for aggregating data types on a genomic scale. *Nature Methods* **11**, 333–337 (2014).
2. Jacobs, G. R. *et al.* Integration of brain and behavior measures for identification of data-driven groups cutting across children with ASD, ADHD, or OCD. *Neuropsychopharmacology* **46**, 643–653 (2021).
3. Blennow, K. *et al.* Traumatic brain injuries. *Nature reviews. Disease primers* **2**, 16084–16084 (2016).
4. Hagler, D. J. *et al.* Image processing and analysis methods for the Adolescent Brain Cognitive Development Study. *NeuroImage* **202**, 116091 (2019).